Rollno :- 2501010212

Q1. Why is NAND gate called universal gate? Justify and implement AND, OR and not gate using NAND. Provide truth table, digital logic Circuit and Boolean expression.

Sol:- A NAND gate is called a universal gate because any basic logic gate (NOT, AND, OR) and consequently any digital logic circuit can be implemented using only NAND gate.

$$Y = \overline{A.B}$$

NAND Gate Truth table:

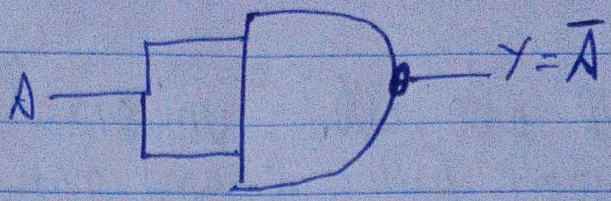
A	B	AND A.B	NAND $Y = \overline{A.B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

A) NOT Gate using NAND

To make a NOT Gate connect both inputs of NAND Gate together.

Scanned with OKEN Scanner

Circuit :-



Boolean expression:

$Y = \overline{A.A}$

Since $= A.A = \overline{A}$

Therefore: $Y = \overline{A}$

Truth Table :-

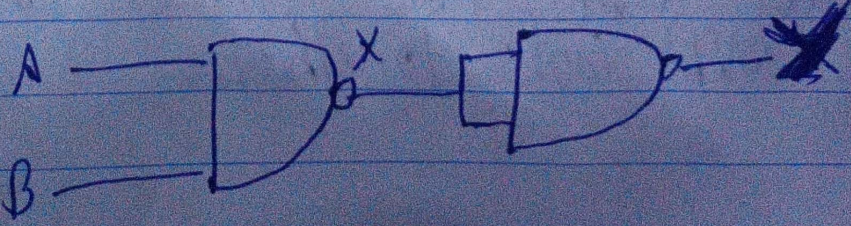
A	Y = A NAND A
0	1
1	0

So, NAND can implement NOT.

B) AND gate using NAND

A NAND gate gives the complement of AND. Therefore, we can connect its output to another NAND gate used as a NOT gate.

Circuit :-



The second NAND has both input connected to X

Boolean expression :-

First NAND ; $X = \overline{A.B}$

Second NAND : $Y = \overline{X.X}$ $(A.A = A)$

Therefore $Y = \overline{X}$

$Y = \overline{\overline{AB}}$

$\boxed{Y = A.B}$

Truth Table :-

A	B	NAND 1	(NAND 2) Final Y
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

So, AND requires 2 NAND gates.

c) OR Gate using NAND

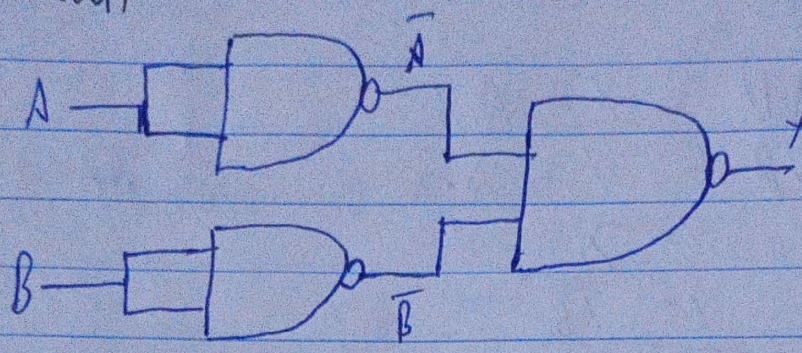
using De Morgan's Theorem

$A + B = \overline{\overline{A}.\overline{B}}$

we can therfore make $\overline{A}$ and $\overline{B}$ using two NAND gates and then feed them into a third NAND gate.

Scanned with OKEN Scanner

Circuit :-



Boolean expression :-

First NAND : $A' = \overline{A}$
second NAND : $B' = \overline{B}$
Third NAND : $Y = \overline{A'.B'}$
using De morgan's Theorm :

$$Y = \overline{\overline{A.B'}}$$
$$Y = \overline{\overline{A}.\overline{B}}$$
$$Y = A + B$$

Truth Table :-

A	B	A'	B'	Final output Y
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

So, OR requires 3 NAND gates.

Scanned with OKEN Scanner

Sop

Q2 Design a Multiplexer (Mux 8 way 16). write the truth table and varify the output for all select lines combination.

Ans = Aim :- To design an 8 way 16 Bit multiplexer (Mux 8 way 16). Write its truth table and varify the output for all possible select-line combinations.

Theory :- A multiplexer (Mux) is a combinational logic circuit that selects one inputs from multiple inputs and forward it to a single output based on the select lines.

A Mux 8 way 16 has :-
8 inputs :- a, b, c, d, e, f, g, h
16 bit width :- each input contains 16 bit.
3 select lines :- sel[2], sel[1], sel[0]
one 16 bit output :- out[16]

Since there are 8 inputs, the number of select lines required is :

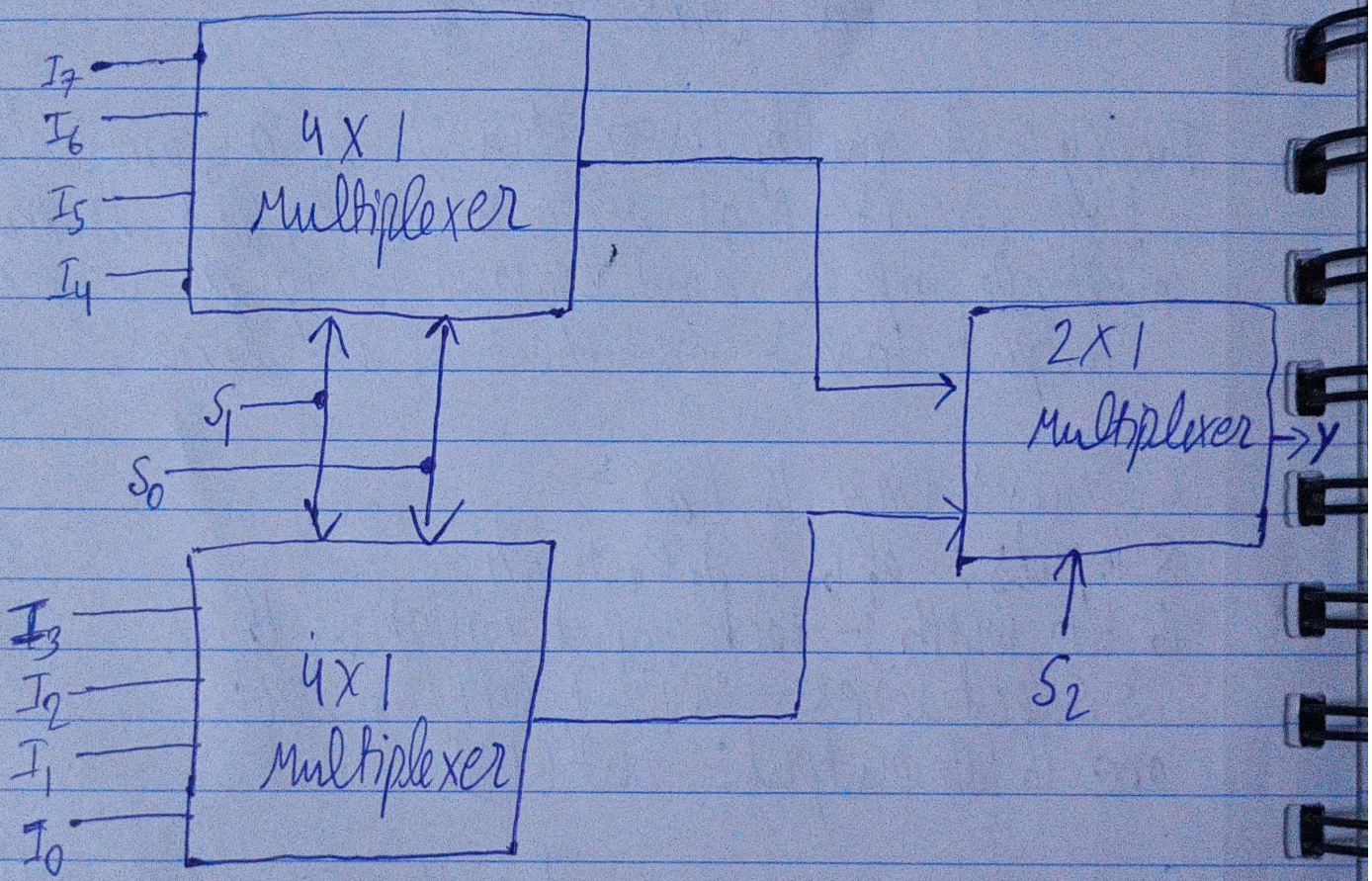
$$2^n = 8$$
$$\Rightarrow \boxed{n = 3}$$

Therefore, the select bits can represent eight combinations, from 000 to 111.

Scanned with OKEN Scanner

Design of 8 way 16 Bit multiplexer (Mux 8way 16)

-> The Mux 8way 16 takes 8 inputs 16 Bit values (I0 - I7) and 3 select lines (S_2, S_1, S_0) to choose one of the inputs and pass it to the 16 bit output Y.



The 8 way 16 Bit multiplexer can be designed using two 4x1 multiplexers and one 2x1 multiplexer.

The first 4x1 multiplexer selects one input from I_0, I_1, I_2, I_3 while the second multiplexer

4×1 selects one inputs from I_4, I_5, I_6 and I_7.

The select lines S_1 and S_0 are connected to both 4×1 multiplexers. Their outputs are then connected to the inputs of the final 2×1 multiplexer.

The third select line S_2 controls the final 2×1 multiplexer.

Therefore:

$S_2 = 0$ selects the output of the lower 4×1 MUX.

$S_1 = 0$ selects the output of the upper 4×1 MUX.

Thus, all three select lines together determine which one of the eight 16 bit input is connected to the output.

Truth Table:-

The selection of the input according to the three select lines as show below:-

S_2	S_1	S_0	selected Input	output
0	0	0	I_0	$Y = I_0$
0	0	1	I_1	$Y = I_1$
0	1	0	I_2	$Y = I_2$
0	1	1	I_3	$Y = I_3$
1	0	0	I_4	$Y = I_4$
1	0	1	I_5	$Y = I_5$
1	1	0	I_6	$Y = I_6$
1	1	1	I_7	$Y = I_7$

Since this is a 16-bit multiplexer, the selected 16 bit input is passed unchanged to 16 bit output.

Boolean Expression:-

Then add -

The Boolean Expression for the 8 way multiplexer is:

$$Y = I_0 \bar{S_2} \bar{S_1} \bar{S_0} + I_1 \bar{S_2} \bar{S_1} S_0 + I_2 \bar{S_2} S_1 \bar{S_0} + I_3 \bar{S_2} S_1 S_0 + I_4 S_2 \bar{S_1} \bar{S_0} + I_5 S_2 \bar{S_1} S_0 + I_6 S_2 S_1 \bar{S_0} + I_7 S_2 S_1 S_0$$

where $\bar{S}$ represents the Not / complement of corresponding select lines.

Verifications:-

To verify the Mux 8 way 16, different combinations of the select lines S_2, S_1 and S_0 are applied. For each combination, the output Y is compared with the corresponding selected input.

Ex:- If distinct 16 bit values are applied to inputs.

select $S_2 S_1 S_0$	selected Input	Expected output
0 0 0	I_0	$Y = I_0$
0 0 1	I_1	$Y = I_1$
0 1 0	I_2	$Y = I_2$
0 1 1	I_3	$Y = I_3$
1 0 0	I_4	$Y = I_4$
1 0 1	I_5	$Y = I_5$
1 1 0	I_6	$Y = I_6$
1 1 1	I_7	$Y = I_7$

Since all possible combinations of the 3 selected lines are given and the output corresponding to the selected input in every case the Mux 8 way 16 operation is verified.

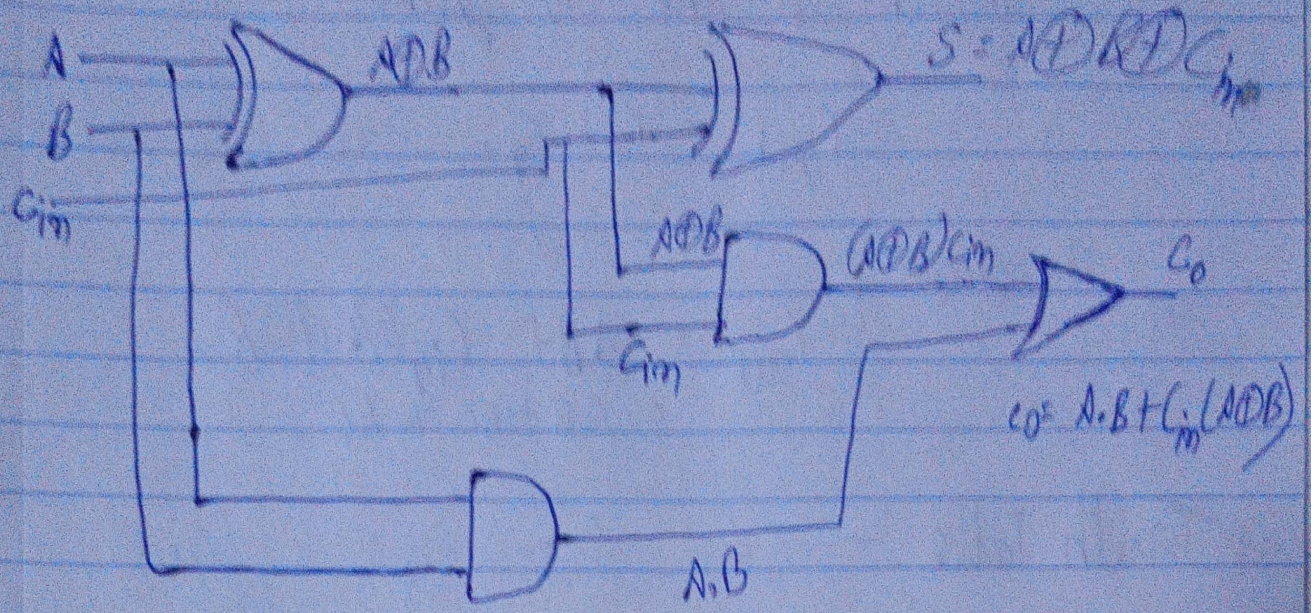
Result:- The 8 way 16 bit multiplexer was successfully designed using two 4x1 multiplexer and one 2x1 multiplexer. The truth table was constructed for all 8 select line combinations, and the output was verified to correspond to select 16-bit input.

Q3 construct a full Adder using two half Adder and each one OR gate. Explain its working and write HDL code. Prove the implementation by comparing the Boolean expression.

sol=> A full Adder is a combinational circuit that adds three 1 bit inputs A, B and Cin and produces two outputs.
Sum and carry.
A full adder can be constructed using two half adders and one OR gate.

Circuit:-
The first Half Adder adds the two input bits A & B, producing an intermediate Sum S_1 and carry C_1. The second half adder takes S_1 and the carry input Cin as its inputs. The carry outputs from both half adders are the connected to an OR gate to produce the

Final carry look.



Working :-

The first half adder adds A and B:

$$S_1 = A \oplus B$$
$$C_1 = A.B$$

The second half adder adds S_1 and C_{in}:

$$Sum = S_1 \oplus C_{in}$$

Therefore

$$Sum = A \oplus B \oplus C_{in}$$

The carry generated by the second half adder is $C_2 = S_1.C_{in}$

The two carry outputs C_1 and C_2 are connected to the OR gate:-

$$C_{out} = C_1 + C_2$$

Therefore;

$$C_{out} = AB + C_{in}(A \oplus B)$$
$$= AB + BC + AC$$

Truth Table :-

A	B	C_{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean expression verification

For the Sum output:

$$Sum = S_1 \oplus C_{in}$$

Since $S_1 = A \oplus B$

Scanned with OKEN Scanner

we get: $Sum = (A \oplus B) \oplus C_{in}$

$Sum = (A \oplus B \oplus C_{in})$

For the carry output:-

$C_{out} = C_1 + C_2$

substituting $C_1 = AB$ and $C_2 = C_{in}(A \oplus B)$:

$C_{out} = AB + C_{in}(A \oplus B)$

using, $A \oplus B = A'B + AB'$

we get: $C_{out} = AB + A'BC_{in} + AB'C_{in}$

This is equivalent to the standard full Adder carry expression:-

$$C_{out} = AB + AC_{in} + BC_{in}$$

Hence, the Boolean expression obtained from the two Half Adders and OR gate match the standard full Adder expression. So, the implementation is verified.

HDL CODE.

```
CHIP Full Adder {
    In a, b, c;
    Out Sum, carry;

    PARTS:
    HalfAdder (a=a, b=b, Sum=Sum1, carry=carry1);
    Half Adder (a=Sum1, b=c, Sum=Sum, carry=carry2);
    OR (a=carry1, b=carry2, out=carry);
}
```

The HDL code uses two Half Adder chips and one OR gate, exactly matching the required full Adder implementation.

Question4: Explain the Hack ALU structure with a block diagram. Explain the role of each output flag. Provide the control bits combination for (x-y) and provide the results for x = 1011 and y = 1100

sol= HACK ALU

The Hack ALU (Arthimetic Logic Unit) is a combinational circuit used in the Hack computer to perform different arthimetic and logical operations on two 16 bit inputs,

Scanned with OKEN Scanner

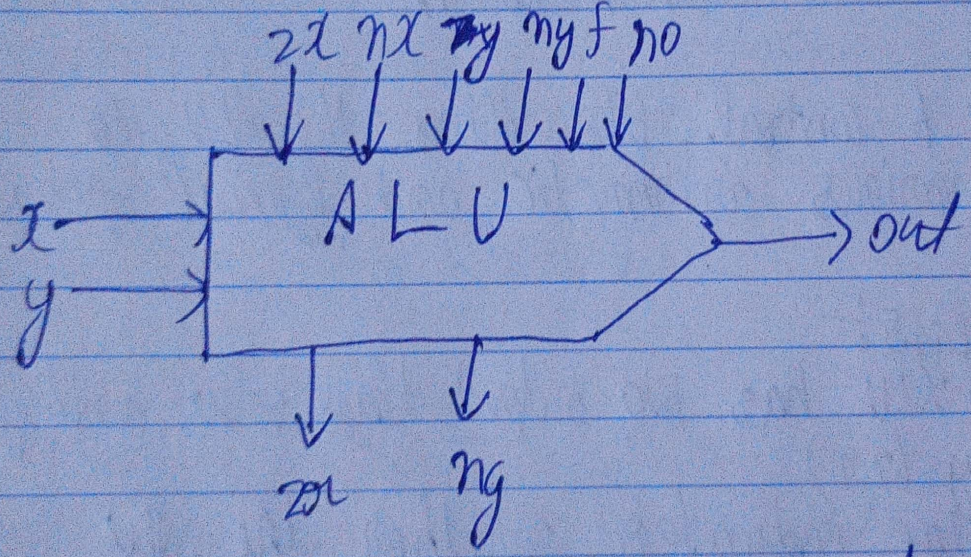
x and y.

The ALU has six control bits:-

zx, nx, zy, ny, f, no.

These control bits determine how the inputs are modified and which operation is performed.

Block Diagram:-



The Hack ALU takes two 16 bit inputs x and y, six control bits, and produces a 16 bit output out. It also produces two output flags zr and ng.

Role of control bits:-

control bit	function
zx	sets x to 0 when zx = 1
nx	negates x when nx = 1
zy	sets y to 0 when zy = 1
ny	negates y when ny = 1
f	selects add when f = 1 otherwise perform AND.
no	negates the final output when no = 1.

Thus the 6 control bits allow the ALU to perform various arithmetic and logical operations.

Output flags:-
The Hack ALU has two output flags: zr and ng.

① Zero flag (zr)
The zr flag indicates whether the ALU output is zero.

zr = 1 if out = 0
zr = 0 if out ≠ 0

for example if

out = 0000000000000000

then:

zr = 1

(2) Negative flag (ng):
→ The ng flag indicates whether the output is negative in 2's compliment representation.

It is equal to the most significant bit of the output; ng = out(15)
So:
ng = 0 → output is non-negative
ng = 1 → output is negative

control bits x-y
The Hack ALU does not have seperate subtraction unit. It performs subtraction using 2's compliment:

$$x - y = x + \bar{y} + 1$$

To perform x-y
keep x unchanged → zx = 0, nx = 0
keep y initially unchanged → zy = 0
negate y → ny = 1
perform addition → f = 1
Do not negative the final result → no = 0

Therefore :-

Control Bit	value
zx	0
nx	0
zy	0
ny	1
f	1
no	0

Hence, the control bit combination is: 000110
in the order: zx nx zy ny f no

Calculation for x = 1011 and y = 1100

Given; x = 1011
y = 1100

we need to calculate: x - y
first find the 2's complement of y.

y = 1100

Invert the bits

$\bar{y}$ = 0011

Now, Add 1:

0011 + 1 = 0100

So: -y = 0100

Now, add x and y

$$\begin{array}{r} 1011 \\ +\ 0100 \\ \hline 1111 \end{array}$$

So: $\boxed{x - y = 1111}$

In 4 bit 2's complement representation

1111 = -1

which is correct because

11 - 12 = -1

for the actual 16 bit Hack ALU, the result would be:

1111111111111111

which also represents -1.

output flags :-

since the output is non-zero:

zr = 0

since the most significant bit is 1:-

ng = 1

final Result:-

parameter	value
x	1011
y	1100
operation	x - y
control bits zx nx zy ny f no	000110

output	1111
Decimal Result	-1
zr	0
ng	1

Thus, the Hack ALU produces -1 for x = 1011 and y = 1100, when the control bits are 000110.

Question: what is the role of the Add16. Design and implement the same and provide HDL code.

Sol:- The Add16 is a chip to add two 16-bit binary numbers. It takes two 16 bit inputs, a[16] and b[16] and produces a 16 bit output out[16].

It performs:

out = A + B

The Add16 is useful in the Hack computer for performing arithmetic operations and is also used as a building block in larger circuits such as the ALU

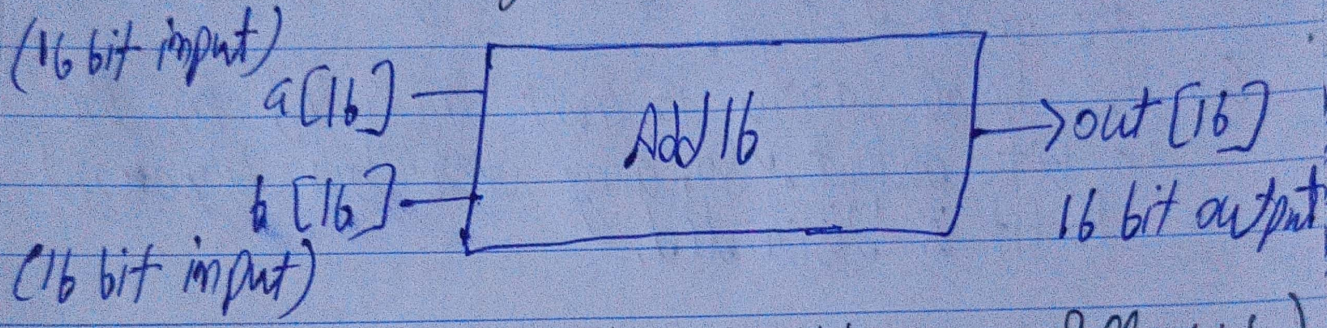
Design and Implementation:-

The Add16 can be implemented using one Half Adder and 15 Full Adders. The least

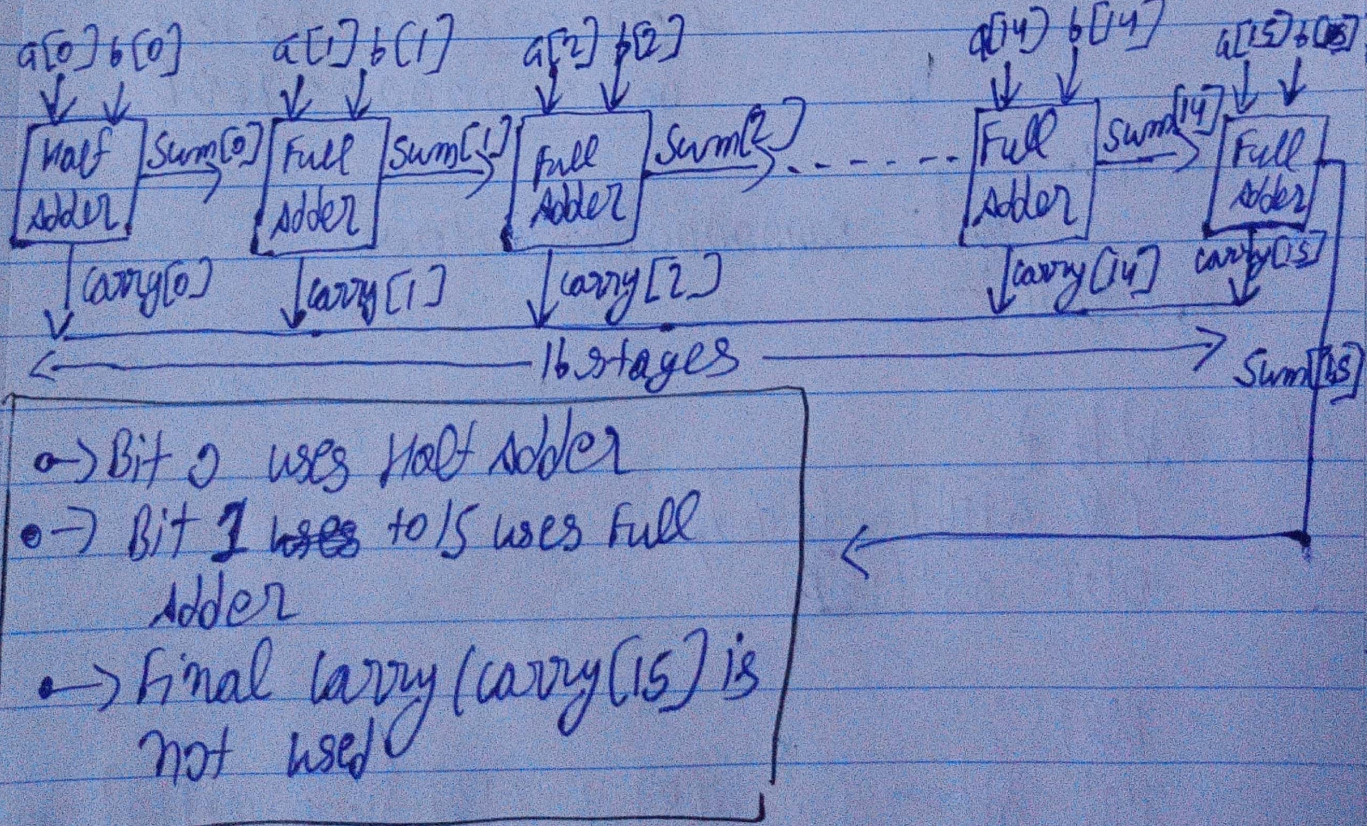
Significant bit (bit 0) does not having a incoming carry, so a Half Adder is used for the first bit. The remaining 15 bit uses full adders, because each recieves the carry generated by the previous stage

Circuit Diagram :-

Add 16 - Block diagram



Internal structure (1 Hack Adder + 15 Full Adder)



Working:-

The working of the circuit is as follows:-

1) The Half Adders add a[0] and b[0] and produces out[0] and the first carry.
2) The next Full Adder adds a[1], b[1] and the carry from the previous stage.
3) This process continues from bit 1 to bit 15.
4) This carry from each stage is passed to the next stage.
5) The final carry is not included in the 16 bit output. So, any overflow beyond 16 bit is discarded.

For example:-

```
a= 0000000000000011
+  0000000000000101
   ----------------
   0000000000001000
```

Hence,

out= 0000000000001000

HDL code:-

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Half Adder (a=a[0], b=b[0], sum=out[0], carry=c);
```

```
FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=c16);
}
```

The final carry c16 is not connected to the output because the Add16 chip produces only a 16 bit result. And carry beyond the 16th bit is discarded.

Thus, Add16 performs 16 bit binary addition by using one half adder followed by 15 full adders in a ripple carry arrangement.